

CS3244 Machine Learning

Scope and notation

These notes develop the main ideas of machine learning from foundational statistical learning and classical models through neural networks and unsupervised learning. Topics are ordered by conceptual dependency, with mathematical definitions, derivations, and model assumptions introduced before applications. This is an unofficial study guide; current course materials determine the assessed scope and notation.

Let the training data be

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d.$$

Write the model as f_{θ} , the prediction as $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$, and the per-example loss as $\ell(y_i, \hat{y}_i)$. Vectors are columns unless stated otherwise. The intercept is written separately unless an augmented design matrix is introduced explicitly.

Task	Output	Typical training objective
Regression	A continuous value or vector	Squared or absolute error
Classification	A class or class probability	Logistic, cross-entropy, or hinge loss
Clustering	A cluster assignment without labels	Within-cluster squared distance
Representation learning	A lower-dimensional or structured representation	Reconstruction loss with structural constraints

Contents

Scope and notation	1
1 Learning Problems and a Leakage-Free Workflow	4
1.1 Training, validation, and test sets	4
1.2 The experimental data flow	4
2 Distance, Scaling, and Nearest Neighbours	5
2.1 Distances and similarities	5
2.2 Feature scaling	5
2.3 The curse of dimensionality	6
2.4 K-nearest neighbours	6
3 Decision Trees and Ensembles	7
3.1 Impurity in classification trees	7
3.2 Regression trees	8
3.3 Stopping and pruning	8

3.4	Bagging and random forests	9
3.5	Boosting and gradient boosting	9
4	Regression and Classification from a Linear Score	10
4.1	Linear regression and the normal equations	10
4.2	Logistic regression	10
4.3	The Perceptron	11
4.4	Maximum margin and hard-margin SVM	11
4.5	Soft-margin SVM and hinge loss	11
4.6	The SVM dual	12
4.7	Kernel SVM	13
5	Learning Model Parameters	13
5.1	Empirical Risk and Regularisation	13
5.2	Least Squares and the Normal Equations	14
5.3	Gradient Descent, Mini-batches, and SGD	14
5.4	Maximum Likelihood Estimation	15
5.5	MAP Estimation and Regularisation	16
6	Generalisation, Regularisation, and Model Selection	16
6.1	Bias–Variance–Noise Decomposition	16
6.2	L_1 and L_2 Geometry	17
6.3	Holdout Validation	18
6.4	K -Fold Cross-Validation	18
6.5	Early Stopping	19
6.6	Correlation in Model Averaging	19
7	Model Evaluation and Class Imbalance	19
7.1	The Confusion Matrix and Basic Metrics	19
7.2	Decision Thresholds and Costs	20
7.3	ROC, Precision–Recall, and AUC	20
7.4	Macro and Micro Averaging	20
7.5	Regression Metrics	21
7.6	Imbalanced Data and Training-Only Resampling	21
8	Feature Representation and Dimensionality Reduction	22
8.1	Consequences of High Dimension	22
8.2	Feature Selection	22
8.3	PCA: An Unsupervised Variance Objective	23
8.4	LDA: A Supervised Separation Objective	23
8.5	Tabular and Temporal Features	24
8.6	Image Features	24
8.7	Text Features	24
9	From the Perceptron to Multilayer Networks	25
9.1	The perceptron as a single neuron	25
9.2	Multilayer perceptrons	25
9.3	Output layers and loss functions	26

9.4	The Softmax-cross-entropy gradient	26
9.5	Forward and backward computation in a two-layer network	27
9.6	Saturation and vanishing gradients	28
10	Convolutional and Recurrent Networks	29
10.1	A convolutional layer computes cross-correlation	29
10.2	Kernel size, stride, padding, and output size	29
10.3	Pooling	30
10.4	Recurrent neural networks	30
10.5	Backpropagation through time	31
10.6	Information flow in an LSTM	31
10.7	Information flow in a GRU	32
11	Unsupervised Learning and Autoencoders	32
11.1	The K-means objective	33
11.2	Initialisation and scope	33
11.3	The autoencoder objective	34
11.4	Linear autoencoders and PCA	35
11.5	Sparse, deep, and convolutional autoencoders	35
11.6	Limits of the compression interpretation	35

1 Learning Problems and a Leakage-Free Workflow

Supervised learning begins with labelled observations

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d.$$

The vector x_i records the d features of example i . In regression, y_i is usually real-valued. In classification, y_i belongs to a finite set; binary labels are commonly encoded as either $\{0, 1\}$ or $\{-1, +1\}$. Each encoding will be stated when it is used.

A model f_θ is determined by parameters θ and produces the prediction $\hat{y} = f_\theta(x)$. A loss $\ell(y, \hat{y})$ measures the error on one example. Training usually minimises the empirical risk

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_\theta(x_i)).$$

The actual objective is low error on new examples from the same data-generating distribution. If (X, Y) denotes such an example, the relevant quantity is the population risk

$$R(\theta) = \mathbb{E}_{(X,Y)} [\ell(Y, f_\theta(X))].$$

The population distribution is unknown, so $R(\theta)$ must be estimated using data that did not determine the fitted model.

1.1 Training, validation, and test sets

A sound experiment splits the data before performing any operation that learns from the data. The three subsets serve different purposes.

1. The training set estimates model parameters, such as linear weights, tree split points, or neural-network weights.
2. The validation set selects model classes and hyperparameters, such as the number of neighbours, tree depth, or the SVM penalty C .
3. The test set is used once, after all choices have been fixed, to estimate the generalisation error of the final procedure.

The word *model* includes the complete preprocessing pipeline. Imputation, standardisation, feature selection, dimensionality reduction, and resampling may all learn properties of a dataset. They must therefore be fitted using only the training data. For example, standardisation uses

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

where μ_j and s_j are estimated from the training set. The same values are then applied to validation and test examples. Estimating them from the full dataset allows information from the test set to enter training and produces an optimistic test error.

After hyperparameters have been selected, the final model may be refitted on the union of the training and validation sets. Every preprocessing parameter must then be re-estimated on that union, while the test set remains untouched. Using test results to revise the model turns the test set into another validation set and invalidates its role as an independent evaluation.

1.2 The experimental data flow

For a fixed family of candidate procedures, the data flow is

$$\mathcal{D}_{\text{train}} \xrightarrow{\text{fit preprocessing and model}} f_\theta \xrightarrow{\mathcal{D}_{\text{val}}} \text{select hyperparameters} \xrightarrow{\text{refit}} f_{\text{final}} \xrightarrow{\mathcal{D}_{\text{test}}} \text{report error}.$$

When the dataset is small, cross-validation within the training data can replace a single validation split. Preprocessing must be refitted inside every fold using only that fold's training portion.

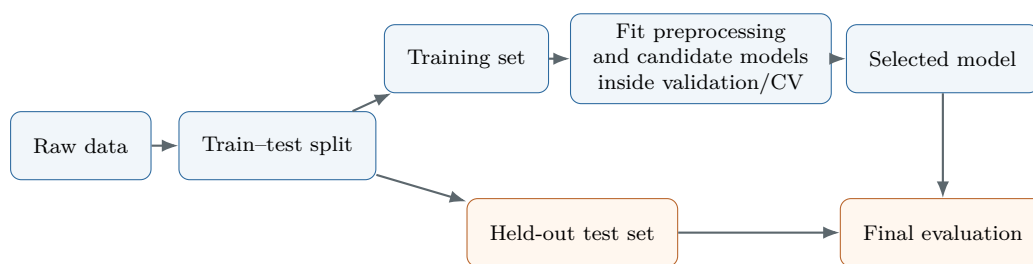


Figure 1: Information boundaries in a training workflow. Scaling, feature selection, and resampling are fitted only on training data or training folds. The test set is used once, after model selection.

2 Distance, Scaling, and Nearest Neighbours

K-nearest neighbours (KNN) assumes that nearby observations tend to have similar labels. Whether this assumption is useful depends first on the feature representation and distance function. Different distances encode different notions of similarity and are not interchangeable.

2.1 Distances and similarities

For $x, z \in \mathbb{R}^d$, the Euclidean and Manhattan distances are

$$d_2(x, z) = \left(\sum_{j=1}^d (x_j - z_j)^2 \right)^{1/2}, \quad d_1(x, z) = \sum_{j=1}^d |x_j - z_j|.$$

Euclidean distance is the straight-line distance and gives relatively large weight to a large discrepancy in one coordinate. Manhattan distance adds absolute coordinate-wise discrepancies and is less dominated by such a discrepancy.

When direction matters more than magnitude, cosine similarity is often more appropriate for non-zero vectors x and z :

$$\cos(x, z) = \frac{x^\top z}{\|x\|_2 \|z\|_2}.$$

The associated cosine distance is $d_{\cos}(x, z) = 1 - \cos(x, z)$. Cosine similarity is not Euclidean distance. After both vectors have been normalised to unit length, however,

$$\left\| \frac{x}{\|x\|_2} - \frac{z}{\|z\|_2} \right\|_2^2 = 2(1 - \cos(x, z)).$$

For sets or binary features, Jaccard similarity ignores features that are absent from both observations. If A and B are the sets of non-zero features, then

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad d_J(A, B) = 1 - J(A, B).$$

This definition is well suited to sparse presence-absence data because a shared zero does not increase similarity. If $A = B = \emptyset$, the displayed ratio is undefined; applications that permit empty sets must state a convention, commonly $J(\emptyset, \emptyset) = 1$.

2.2 Feature scaling

Distances are sensitive to units. If one feature is measured in thousands and another lies between 0 and 1, the first feature can dominate every Euclidean distance. Two common transformations are standardisation

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}$$

and min–max scaling

$$z_{ij} = \frac{x_{ij} - a_j}{b_j - a_j},$$

where a_j and b_j are the minimum and maximum of feature j in the training set. Standardisation does not confine values to a fixed interval, but it is usually less dependent on two observed endpoints. In either case, all statistics are estimated from the training data alone. A constant feature has $s_j = 0$ and $b_j - a_j = 0$; remove it or map it to a fixed value rather than divide by zero.

Scaling is not an automatic requirement. Binary indicators, variables with a meaningful physical scale, and inputs to decision trees need not receive the same treatment. The choice follows from how the model uses each feature.

2.3 The curse of dimensionality

With a fixed number of observations, the sample becomes sparse as the dimension increases. In a unit hypercube, a smaller hypercube of side length r occupies the fraction r^d of the total volume. To enclose a fixed fraction q of the volume, its side length must be

$$r = q^{1/d}.$$

As d increases, r approaches 1. A neighbourhood must therefore extend through much of every coordinate range before it contains a fixed fraction of the observations; it is no longer genuinely local.

Many high-dimensional distributions also exhibit distance concentration: the relative difference between the nearest and farthest distances shrinks. The distances remain computable, but their ranking carries less information. For KNN, high dimensionality primarily causes sparse neighbourhoods and distance concentration, which can in turn increase variance. More observations, feature selection, dimensionality reduction, or a structured similarity measure may be needed.

2.4 K-nearest neighbours

For a query x , KNN finds the k training observations with the smallest distance and denotes their index set by $N_k(x)$. A classifier uses a majority vote,

$$\hat{y} = \arg \max_c \sum_{i \in N_k(x)} \mathbf{1}(y_i = c),$$

whereas a regressor commonly uses the mean

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i.$$

Closer observations can be given more weight. One possible choice is

$$w_i(x) = \frac{1}{d(x, x_i) + \varepsilon}, \quad \hat{y} = \frac{\sum_{i \in N_k(x)} w_i(x) y_i}{\sum_{i \in N_k(x)} w_i(x)}.$$

where $\varepsilon > 0$. The same weights can be used for a weighted class vote.

The value of k controls smoothness. A small k preserves local structure but is sensitive to noise and individual outliers. A large k is more stable but may erase a genuine boundary. The rule $k = \sqrt{n}$ is at most an initial heuristic; validation or cross-validation must determine the final value. An odd k reduces ties in binary classification, although ties can still occur with several classes or repeated distances and therefore require an explicit tie-breaking rule.

KNN has little explicit training cost but stores the training set. A brute-force implementation computes n distances in d dimensions for each query, giving time $O(nd)$ and storage $O(nd)$. Tree-based indices can accelerate low-dimensional queries, but their benefit often declines in high dimensions.

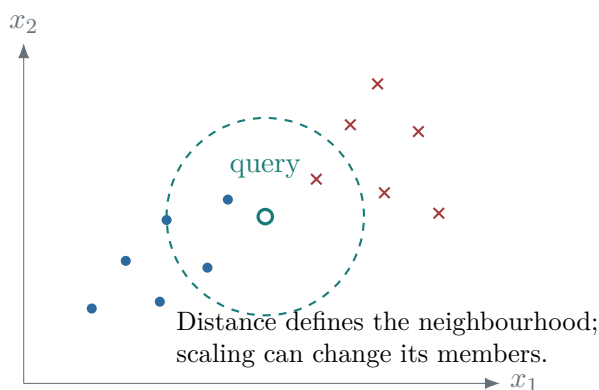


Figure 2: KNN searches for k observations around a query. The circle represents one neighbourhood under a chosen distance scale; neighbourhoods become sparse in high dimensions.

3 Decision Trees and Ensembles

A decision tree partitions the input space recursively. Each internal node asks a question about a feature, and each leaf stores a prediction. For a continuous feature, a common split is

$$x_j \leq t,$$

where j selects a feature and t is a threshold. For the observations S at the current node, the split produces

$$S_L = \{(x_i, y_i) \in S : x_{ij} \leq t\}, \quad S_R = S \setminus S_L.$$

The two subsets need not have equal size. Training selects the split that most reduces label impurity or numerical dispersion and repeats the process at the child nodes.

3.1 Impurity in classification trees

Let p_c be the proportion of class c at node S . Entropy and Gini impurity are

$$H(S) = - \sum_{c=1}^C p_c \log_2 p_c, \quad G(S) = 1 - \sum_{c=1}^C p_c^2,$$

with the convention $0 \log 0 = 0$. Both are zero at a pure node and increase as the class distribution becomes more mixed.

Suppose a split produces child nodes of sizes n_L and n_R , where $n_S = n_L + n_R$. For an impurity measure I , the reduction is

$$\Delta I = I(S) - \frac{n_L}{n_S} I(S_L) - \frac{n_R}{n_S} I(S_R).$$

When $I = H$, this reduction is the information gain. A greedy tree normally chooses the feature and threshold with the largest ΔI .

For example, a node containing six positive and four negative examples has entropy

$$H(S) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.971.$$

Suppose a split sends four positive examples to the left and the remaining two positive and four negative examples to the right. The weighted child entropy is

$$\frac{4}{10} \cdot 0 + \frac{6}{10} \left(-\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \right) \approx 0.551,$$

so the information gain is approximately 0.420. The weighting accounts for both child purity and child size; inspecting only one child is insufficient.

A classification leaf can output its majority class or estimated class proportions,

$$\hat{P}(Y = c | x) = \frac{n_c}{n_{\text{leaf}}}.$$

The latter permits threshold adjustment, although probabilities estimated from small leaves are unstable.

3.2 Regression trees

A regression tree uses the same partitioning mechanism. A leaf predicts its mean response

$$\bar{y}_S = \frac{1}{|S|} \sum_{i \in S} y_i.$$

A common node impurity is the within-node sum of squared errors

$$I_{\text{reg}}(S) = \sum_{i \in S} (y_i - \bar{y}_S)^2.$$

Training selects the split that maximises

$$I_{\text{reg}}(S) - I_{\text{reg}}(S_L) - I_{\text{reg}}(S_R).$$

The fitted function is constant within each leaf region, so a single regression tree produces a piecewise-constant predictor.

3.3 Stopping and pruning

If a tree is allowed to keep splitting, a leaf may contain only one observation. The resulting tree has low training error and high variance. Pre-pruning limits growth directly through a maximum depth, a minimum leaf size, or a minimum impurity reduction. Post-pruning first grows a larger tree and then removes branches whose improvement is not supported by validation data.

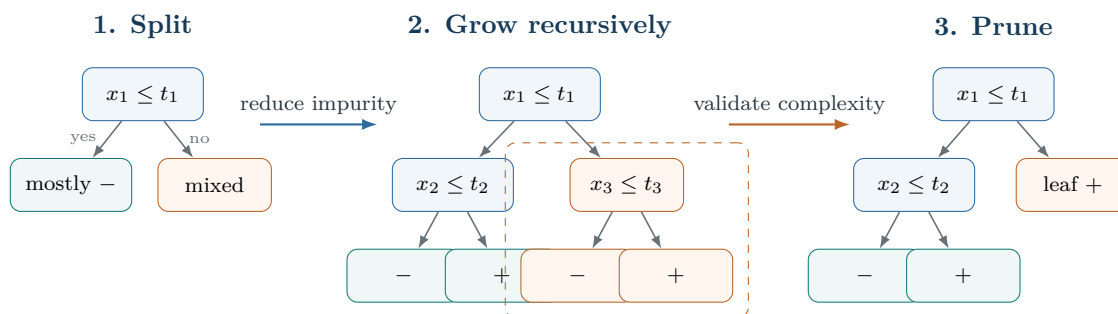


Figure 3: A tree chooses a locally useful split, recursively splits eligible non-pure children while the pre-pruning conditions permit further growth, and may then prune a branch whose added complexity is not supported by validation data. Pruning replaces the highlighted subtree by one leaf.

Cost-complexity pruning penalises the number of leaves:

$$R_\alpha(T) = R(T) + \alpha |\mathcal{L}(T)|,$$

where $R(T)$ is the training loss of tree T , $\mathcal{L}(T)$ is its set of leaves, and $\alpha \geq 0$. Larger values of α favour smaller trees. The penalty is selected using validation data.

A prediction visits only the path from the root to a leaf, so its cost is $O(h)$ for tree depth h . A balanced tree can have logarithmic depth, whereas a degenerate tree can have depth linear in its number of leaves. Training cost depends on how candidate thresholds are generated and sorted; it cannot be described by a universal $O(dn^2 \log n)$ expression.

3.4 Bagging and random forests

A deep tree can fit a complicated boundary, but small changes in the training sample may alter its upper splits. Bagging constructs M bootstrap samples. Each contains n observations drawn with replacement from the original sample. The base learners are fitted separately, then averaged in regression or combined by majority vote or mean class probability in classification:

$$\bar{f}(x) = \frac{1}{M} \sum_{m=1}^M f_m(x).$$

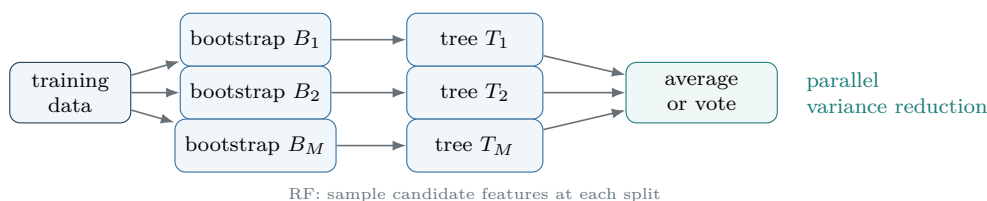
Suppose the prediction of each base learner at a fixed x has variance σ^2 , and any pair has approximately the same correlation ρ . Then

$$\text{Var}[\bar{f}(x)] = \sigma^2 \left(\rho + \frac{1 - \rho}{M} \right). \quad (1)$$

The variance becomes σ^2/M only when the predictors are independent ($\rho = 0$). Trees trained on related bootstrap samples remain correlated, so the reduction is smaller and depends on how effectively the procedure decorrelates them.

A random forest adds feature randomisation. At each node, the tree searches for a split among only m_{try} randomly selected features. This prevents every tree from repeatedly relying on the same strong feature. The setting $m_{\text{try}} \approx \sqrt{d}$ is a common classification default, not a theoretical optimum; regression implementations often use a different default. The feature count, tree depth, and leaf size remain hyperparameters.

Bagging / random forest



Gradient boosting

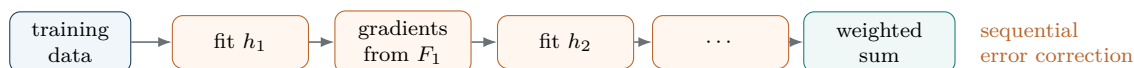


Figure 4: Bagging fits learners independently on bootstrap samples and aggregates their predictions; a random forest additionally randomises candidate features at each split. Boosting is sequential: each learner follows the loss gradients of the current additive model.

3.5 Boosting and gradient boosting

Bagged learners can be trained in parallel. Boosting adds learners sequentially, so that each new learner addresses errors of the current ensemble. An additive model has the form

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta \gamma_m h_m(x),$$

where h_m is often a shallow tree and $\eta \in (0, 1]$ is the learning rate.

Gradient boosting applies this construction to a differentiable loss $\ell(y, F(x))$. At iteration m , it computes the negative gradient at each training observation,

$$r_i^{(m)} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}},$$

and fits h_m to $\{(x_i, r_i^{(m)})\}$. A line search may choose

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n \ell(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)),$$

followed by

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m h_m(x).$$

For squared loss $\ell(y, F) = \frac{1}{2}(y - F)^2$, the negative gradient is the residual $y_i - F_{m-1}(x_i)$. Fitting residuals is therefore the squared-loss instance of gradient boosting, not its definition for every loss.

Boosting typically reduces bias, while tree depth, learning rate, and the number of rounds determine capacity and variance. Select the number of rounds by validation or early stopping.

4 Regression and Classification from a Linear Score

Several linear models share the score

$$s(x) = w^\top x + b.$$

They differ in how they interpret the score, which loss they minimise, and whether they impose a margin. Linear regression uses $s(x)$ as a numerical prediction. Logistic regression maps it to a probability. The Perceptron and SVM classify according to its sign.

4.1 Linear regression and the normal equations

Linear regression takes the score itself as the prediction, $\hat{y}_i = s(x_i)$. With the intercept absorbed into an augmented design matrix X and parameter vector θ , its squared-error objective is

$$L(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2.$$

The derivation of the normal equations, the rank condition for the inverse formula, and the regularised solution are given in Section 5.2. Linear regression uses the score directly; unlike the classifiers below, it neither thresholds the score nor maps it to a probability.

4.2 Logistic regression

Let $y_i \in \{0, 1\}$. Logistic regression maps the linear score to a probability:

$$p_i = P(Y = 1 \mid x_i) = \sigma(s_i), \quad s_i = w^\top x_i + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Consequently,

$$P(Y = y_i \mid x_i) = p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Under conditional independence of the observations, maximum conditional likelihood is equivalent to minimising the mean negative log-likelihood

$$\begin{aligned} L(w, b) &= -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] \\ &= \frac{1}{n} \sum_{i=1}^n [\log(1 + e^{s_i}) - y_i s_i]. \end{aligned}$$

This is binary cross-entropy. With the augmented design matrix X and $p = (p_1, \dots, p_n)^\top$, its gradient is

$$\nabla_{\theta} L = \frac{1}{n} X^\top (p - y).$$

The optimisation procedure is developed in Section 5.2 and the gradient-descent section that follows it.

The threshold $p_i \geq 0.5$ is equivalent to $s_i \geq 0$, so the decision boundary is $w^\top x + b = 0$. Changing the probability threshold changes the classifier without refitting the probability model. With labels $y_i \in \{-1, +1\}$, the same loss is

$$\ell_{\log}(y_i, s_i) = \log(1 + e^{-y_i s_i}).$$

4.3 The Perceptron

The Perceptron uses labels $y_i \in \{-1, +1\}$ and predicts

$$\hat{y}_i = \text{sign}(w^\top x_i + b).$$

An observation is correctly classified when $y_i s_i > 0$. On a misclassified observation, the update is

$$w \leftarrow w + \eta y_i x_i, \quad b \leftarrow b + \eta y_i.$$

This update is a subgradient step on the Perceptron loss

$$\ell_{\text{perc}}(y_i, s_i) = \max(0, -y_i s_i).$$

The loss becomes zero as soon as the sign is correct and does not reward a wider margin. For linearly separable training data, the Perceptron reaches a separating hyperplane after finitely many updates. It need not find the maximum-margin hyperplane and does not directly produce class probabilities. On non-separable data, the original algorithm may continue updating without convergence.

4.4 Maximum margin and hard-margin SVM

Many separating hyperplanes may exist for linearly separable data. SVM fixes a functional-margin scale and selects the hyperplane with the largest geometric margin. The hard-margin primal problem is

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|_2^2 \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

The signed distance from x_i to $w^\top x + b = 0$ is $(w^\top x_i + b)/\|w\|_2$. Under the constraints, the closest training points lie on $w^\top x + b = \pm 1$, and the distance between these support hyperplanes is

$$\frac{2}{\|w\|_2}.$$

Minimising $\frac{1}{2} \|w\|_2^2$ therefore maximises the margin. The hard-margin problem is feasible only when the training data are linearly separable.

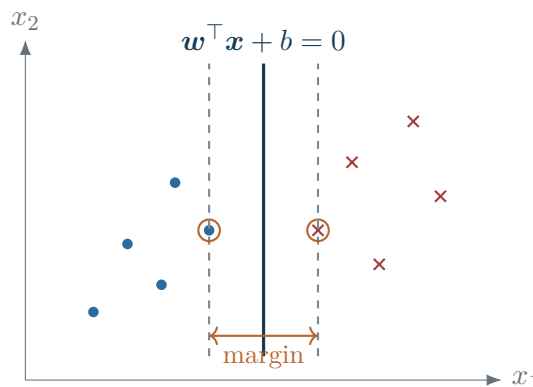


Figure 5: Logistic regression, the perceptron, and SVM use the same linear score $z = w^\top x + b$. SVM additionally chooses a boundary with maximum geometric margin.

4.5 Soft-margin SVM and hinge loss

Slack variables $\xi_i \geq 0$ allow margin violations, with $C > 0$ controlling their penalty:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \end{aligned}$$

For fixed w, b , the optimal slack is

$$\xi_i = \max(0, 1 - y_i(w^\top x_i + b)).$$

The primal problem is therefore equivalent to

$$\min_{w, b} \quad \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \underbrace{\max(0, 1 - y_i s_i)}_{\text{hinge loss}}.$$

A large C penalises violations heavily and favours fitting more training observations. A small C permits more violations in exchange for a wider margin. Another common convention writes the objective as $\lambda \|w\|_2^2/2 + (1/n) \sum_i \ell_i$. The two conventions correspond under a change of scale; values of C and λ cannot be compared without specifying the objective.

Logistic regression, the Perceptron, and SVM all depend on the signed margin ys , but use different losses:

$$\begin{aligned} \ell_{\log}(y, s) &= \log(1 + e^{-ys}), \\ \ell_{\text{perc}}(y, s) &= \max(0, -ys), \\ \ell_{\text{hinge}}(y, s) &= \max(0, 1 - ys). \end{aligned}$$

Logistic loss is smooth and admits a probability interpretation. Perceptron loss vanishes once the sign is correct. Hinge loss requires a functional margin of at least 1.

4.6 The SVM dual

The dual form of soft-margin SVM depends on the inputs only through inner products. Introduce multipliers $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$. The Lagrangian is

$$\begin{aligned} \mathcal{L}(w, b, \xi, \alpha, \mu) &= \frac{1}{2} \|w\|_2^2 + C \sum_i \xi_i \\ &\quad - \sum_i \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i. \end{aligned}$$

Stationarity with respect to the primal variables gives

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} = 0 &\implies w = \sum_i \alpha_i y_i x_i, \\ \frac{\partial \mathcal{L}}{\partial b} = 0 &\implies \sum_i \alpha_i y_i = 0, \quad \frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies C - \alpha_i - \mu_i = 0. \end{aligned}$$

Since $\mu_i \geq 0$, the last relation yields $0 \leq \alpha_i \leq C$. Substitution into the Lagrangian gives the dual problem

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The hard-margin dual has the same objective and equality constraint, but only $\alpha_i \geq 0$ and no upper bound C .

Observations with $\alpha_i > 0$ appear in $w = \sum_i \alpha_i y_i x_i$ and are the support vectors. If $0 < \alpha_i < C$, complementary slackness gives

$$y_i(w^\top x_i + b) = 1.$$

Hence any such support vector gives

$$b = y_i - \sum_j \alpha_j y_j x_j^\top x_i.$$

Numerical implementations commonly average this value over several support vectors satisfying $0 < \alpha_i < C$.

4.7 Kernel SVM

Both the dual objective and the prediction function require only inner products. If a feature map ϕ exists, a kernel

$$K(x, z) = \phi(x)^\top \phi(z)$$

can replace the original inner product without explicitly constructing the possibly high-dimensional vector $\phi(x)$. The dual becomes

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(x_i, x_j) \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0, \end{aligned}$$

and the prediction is

$$\hat{y}(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i K(x_i, x) + b \right).$$

Only support vectors with $\alpha_i > 0$ contribute to this sum.

Common kernels include the linear kernel

$$K_{\text{linear}}(x, z) = x^\top z,$$

the polynomial kernel

$$K_{\text{poly}}(x, z) = (\gamma x^\top z + r)^q, \quad \gamma > 0, \quad r \geq 0, \quad q \in \{1, 2, \dots\},$$

and the radial basis function kernel

$$K_{\text{RBF}}(x, z) = \exp(-\gamma \|x - z\|_2^2), \quad \gamma > 0.$$

For a valid kernel, the Gram matrix $K_{ij} = K(x_i, x_j)$ must be positive semidefinite for every finite set of inputs. In the RBF kernel, a larger γ gives each observation a narrower region of influence and usually permits a more intricate boundary; a smaller γ gives a smoother boundary. The values of C and γ jointly control model capacity and should be selected by the leakage-free validation procedure described earlier.

5 Learning Model Parameters

A model specifies a family of functions indexed by parameters θ . Training selects one function from this family. Section 1 defines population and empirical risk and explains why training fit alone does not measure generalisation. Here the observations are assumed iid and drawn from the deployment distribution unless stated otherwise; time dependence, grouping, or distribution shift requires a different split and error analysis.

5.1 Empirical Risk and Regularisation

Regularised training adds a parameter preference to empirical risk:

$$J(\theta) = \hat{R}_n(\theta) + \lambda \Omega(\theta), \quad \lambda \geq 0.$$

The penalty Ω restricts effective model complexity, while λ controls its strength relative to data fit.

5.2 Least Squares and the Normal Equations

For linear regression, let $X \in \mathbb{R}^{n \times d}$ have row $\mathbf{x}_i^\top$ and let $\mathbf{y} = (y_1, \dots, y_n)^\top \in \mathbb{R}^n$. To include an intercept, define

$$\tilde{X} = [\mathbf{1} \ X] \in \mathbb{R}^{n \times (d+1)}, \quad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \in \mathbb{R}^{d+1}.$$

Then $f(\mathbf{x}) = b + \mathbf{w}^\top \mathbf{x}$, and the mean squared error is

$$L(\tilde{\boldsymbol{\theta}}) = \frac{1}{n} \|\tilde{X}\tilde{\boldsymbol{\theta}} - \mathbf{y}\|_2^2.$$

Expanding the quadratic gives

$$\begin{aligned} L(\tilde{\boldsymbol{\theta}}) &= \frac{1}{n} (\tilde{X}\tilde{\boldsymbol{\theta}} - \mathbf{y})^\top (\tilde{X}\tilde{\boldsymbol{\theta}} - \mathbf{y}) \\ &= \frac{1}{n} (\tilde{\boldsymbol{\theta}}^\top \tilde{X}^\top \tilde{X} \tilde{\boldsymbol{\theta}} - 2\tilde{\boldsymbol{\theta}}^\top \tilde{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}), \\ \nabla_{\tilde{\boldsymbol{\theta}}} L(\tilde{\boldsymbol{\theta}}) &= \frac{2}{n} \tilde{X}^\top (\tilde{X}\tilde{\boldsymbol{\theta}} - \mathbf{y}). \end{aligned}$$

Setting the gradient to zero yields the normal equations

$$\tilde{X}^\top \tilde{X} \hat{\tilde{\boldsymbol{\theta}}} = \tilde{X}^\top \mathbf{y}.$$

If $\tilde{X}$ has full column rank, so that $\text{rank}(\tilde{X}) = d + 1$, then $\tilde{X}^\top \tilde{X}$ is invertible and the unique solution is

$$\hat{\tilde{\boldsymbol{\theta}}} = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top \mathbf{y}.$$

If $\tilde{X}$ is rank deficient, the least-squares solution need not be unique and this inverse must not be used. The minimum-norm solution is $\tilde{X}^+ \mathbf{y}$, where $\tilde{X}^+$ is the Moore–Penrose pseudoinverse. Ridge regularisation provides another remedy by making the system matrix invertible.

5.3 Gradient Descent, Mini-batches, and SGD

Closed-form solutions are available for only a limited class of objectives. A differentiable objective $J(\boldsymbol{\theta})$ can instead be minimised iteratively. Gradient descent evaluates the gradient at the current iterate:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla J(\boldsymbol{\theta}_t),$$

where $\eta_t > 0$ is the learning rate. The negative gradient is the direction of steepest local decrease under the Euclidean norm because

$$J(\boldsymbol{\theta}_t + \Delta \boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_t) + \nabla J(\boldsymbol{\theta}_t)^\top \Delta \boldsymbol{\theta}.$$

The gradient is evaluated at $\boldsymbol{\theta}_t$, not at the unknown $\boldsymbol{\theta}_{t+1}$. A learning rate that is too large can overshoot low-loss regions or cause divergence; a learning rate that is too small gives slow progress.

When the empirical objective decomposes as $J(\boldsymbol{\theta}) = n^{-1} \sum_i \ell_i(\boldsymbol{\theta})$, a full-batch update reads every observation. For a randomly sampled mini-batch $B_t \subseteq \{1, \dots, n\}$, define

$$\mathbf{g}_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell_i(\boldsymbol{\theta}_t), \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t.$$

Under uniform sampling,

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\theta}_t] = \nabla J(\boldsymbol{\theta}_t).$$

The case $|B_t| = 1$ is stochastic gradient descent (SGD); values between 1 and n give mini-batch gradient descent. Small batches reduce the cost of an update but produce a noisier gradient estimate. Large batches give a more stable direction at a higher per-update cost. Stochastic updates can sometimes move away from flat saddle regions, but they do not guarantee a global optimum.

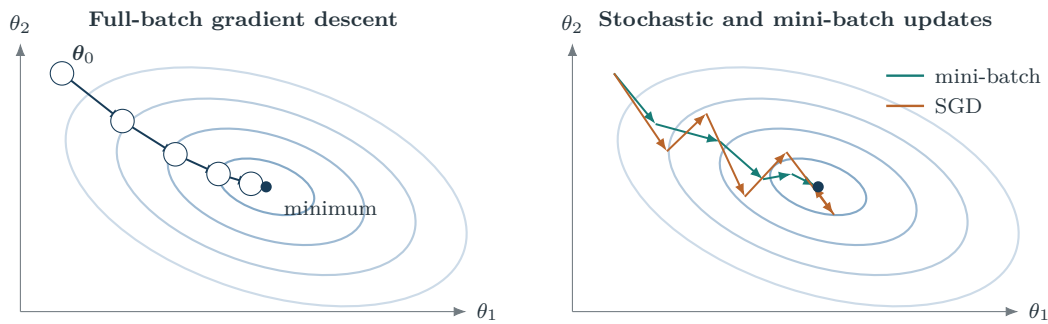


Figure 6: Optimisation paths on the same schematic loss contours. A full-batch gradient gives a deterministic direction for fixed parameters. Mini-batch and single-example gradients are random estimates of that direction; smaller batches usually produce larger step-to-step variation, even when the estimate is unbiased.

For an L -smooth convex objective, full-batch gradient descent converges in objective value under the fixed-step condition $0 < \eta < 2/L$. Convergence claims for SGD additionally require assumptions on sampling, gradient variance, and usually a decreasing step schedule. Neural-network objectives are generally non-convex; optimisation then seeks a low-loss stationary region, while validation measures generalisation.

5.4 Maximum Likelihood Estimation

A probabilistic model specifies a conditional distribution $p_{\theta}(y \mid \mathbf{x})$. Conditional independence of the labels gives the likelihood

$$p_{\theta}(\mathbf{y} \mid X) = \prod_{i=1}^n p_{\theta}(y_i \mid \mathbf{x}_i).$$

Maximum likelihood estimation (MLE) is equivalent to minimising negative log-likelihood:

$$\begin{aligned} \hat{\theta}_{\text{MLE}} &\in \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(y_i \mid \mathbf{x}_i) \\ &= \arg \min_{\theta} \left[- \sum_{i=1}^n \log p_{\theta}(y_i \mid \mathbf{x}_i) \right]. \end{aligned}$$

The logarithm turns a product into a sum and avoids numerical underflow from multiplying many small probabilities.

Squared error follows from a Gaussian noise model. Assume

$$y_i = f_{\theta}(\mathbf{x}_i) + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2),$$

with a variance that does not depend on $\mathbf{x}_i$. Then

$$-\log p_{\theta}(\mathbf{y} \mid X) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - f_{\theta}(\mathbf{x}_i))^2.$$

For fixed σ^2 , the first term is constant in θ , and the positive scale of the second term does not change its minimiser. MLE under independent homoscedastic Gaussian noise therefore gives the same estimate as least squares. If the conditional variance changes with the input, ordinary MSE is no longer the complete negative log-likelihood.

5.5 MAP Estimation and Regularisation

Maximum a posteriori estimation (MAP) places a prior distribution $p(\boldsymbol{\theta})$ on the parameters:

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{\text{MAP}} &\in \arg \max_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D}) \\ &= \arg \max_{\boldsymbol{\theta}} [\log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})].\end{aligned}$$

The evidence $p(\mathcal{D})$ is independent of $\boldsymbol{\theta}$ and therefore does not affect the maximiser.

Suppose the parameter components are independent and follow zero-mean Gaussian priors:

$$\theta_j \sim \mathcal{N}(0, \tau^2), \quad -\log p(\boldsymbol{\theta}) = \frac{1}{2\tau^2} \|\boldsymbol{\theta}\|_2^2 + \text{constant}.$$

The resulting MAP objective has an L_2 penalty. Independent Laplace priors instead give

$$p(\theta_j) = \frac{1}{2b} \exp\left(-\frac{|\theta_j|}{b}\right), \quad -\log p(\boldsymbol{\theta}) = \frac{1}{b} \|\boldsymbol{\theta}\|_1 + \text{constant},$$

which produces an L_1 penalty. The regularisation coefficient depends on both the likelihood scale and the prior scale. Replacing a summed loss by a mean loss also changes the corresponding numerical value of λ .

These correspondences require the stated independent, zero-centred priors. The intercept is commonly left unregularised because an overall shift should not be treated as a feature effect. MAP returns the posterior mode; it does not retain the uncertainty represented by the full posterior distribution.

6 Generalisation, Regularisation, and Model Selection

A training algorithm observes a finite sample but must predict on new inputs. Generalisation analysis treats the training set itself as random. Different samples produce different fitted functions, so test error contains systematic bias, sensitivity to the training sample, and noise in the data-generating process.

6.1 Bias–Variance–Noise Decomposition

Consider regression with squared loss. Define the regression function and conditional noise variance by

$$f^*(\mathbf{x}) = \mathbb{E}[Y \mid X = \mathbf{x}], \quad \sigma^2(\mathbf{x}) = \text{Var}(Y \mid X = \mathbf{x}).$$

Equivalently,

$$Y = f^*(\mathbf{x}) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid \mathbf{x}] = 0, \quad \text{Var}(\varepsilon \mid \mathbf{x}) = \sigma^2(\mathbf{x}).$$

Let a random training set $\mathcal{D}$ produce the fitted function $\hat{f}_{\mathcal{D}}$, and define its average prediction over possible training sets by

$$\bar{f}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})].$$

At a fixed input $\mathbf{x}$, the expected test error is

$$\begin{aligned}\mathbb{E}_{\mathcal{D}, Y \mid \mathbf{x}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - Y)^2] \\ &= \mathbb{E}_{\mathcal{D}, Y \mid \mathbf{x}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}) - \varepsilon)^2] \\ &= \mathbb{E}_{\mathcal{D}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2] + \sigma^2(\mathbf{x}).\end{aligned}$$

The cross term vanishes because $\mathbb{E}[\varepsilon \mid \mathbf{x}] = 0$ and test-label noise is independent of the training set. Add and subtract $\bar{f}(\mathbf{x})$ in the remaining expectation:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - f^*(\mathbf{x}))^2] \\ &= \mathbb{E}_{\mathcal{D}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - f^*(\mathbf{x}))^2] \\ &= \underbrace{\mathbb{E}_{\mathcal{D}} [(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \bar{f}(\mathbf{x}))^2]}_{\text{variance}} + \underbrace{(\bar{f}(\mathbf{x}) - f^*(\mathbf{x}))^2}_{\text{squared bias}}.\end{aligned}$$

The second cross term is zero because $\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x}) - \bar{f}(\mathbf{x})] = 0$. Hence

$$\mathbb{E}_{\mathcal{D}, Y|\mathbf{x}} \left[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - Y)^2 \right] = \text{Bias}^2(\mathbf{x}) + \text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(\mathbf{x})) + \sigma^2(\mathbf{x}).$$

Taking an additional expectation over X gives the decomposition of the overall test MSE. Bias measures the systematic difference between the average prediction and f^* . Variance measures sensitivity to the sampled training set. The term $\sigma^2(\mathbf{x})$ is irreducible when predicting an individual response under squared loss. Bias is not irreducible noise. This exact identity is specific to squared loss; the same terms are used only qualitatively for most classification losses.

Increasing model flexibility often reduces bias and increases variance, but this is not a theorem that applies pointwise to every model family. The shape of the trade-off also depends on the optimiser, regularisation, and data distribution.

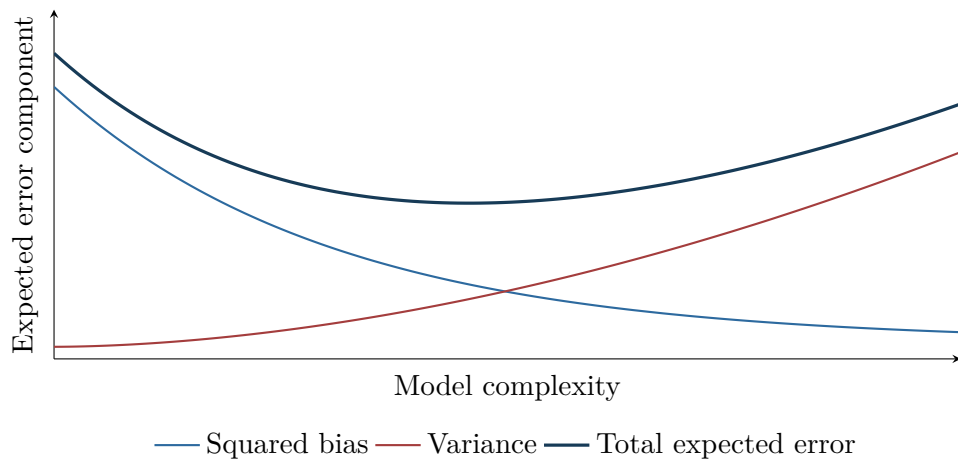


Figure 7: As model complexity grows, bias often falls while variance rises. These curves describe a common pattern rather than a theorem for every model family; irreducible noise shifts the total error upward.

6.2 L_1 and L_2 Geometry

Two common regularised objectives are

$$\min_{\boldsymbol{\theta}} \hat{R}_n(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2, \quad \min_{\boldsymbol{\theta}} \hat{R}_n(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1.$$

The L_2 penalty is smooth and encourages a smaller coefficient norm. In ridge regression, individual coordinates need not decrease monotonically when features are correlated. The penalty does not, by geometry alone, make many coefficients exactly zero. The L_1 ball has corners on the coordinate axes. A loss contour that first touches this boundary often does so at a corner, which produces a sparse solution. The L_1 norm is non-differentiable when a coordinate is zero, but it is convex and can be optimised with subgradients or coordinate descent.

The penalised and constrained forms are

$$\min_{\boldsymbol{\theta}} \hat{R}_n(\boldsymbol{\theta}) + \lambda \Omega(\boldsymbol{\theta}), \quad \min_{\boldsymbol{\theta}} \hat{R}_n(\boldsymbol{\theta}) \quad \text{subject to} \quad \Omega(\boldsymbol{\theta}) \leq C.$$

Under convexity, feasibility, and strong-duality conditions, a solution to the penalised problem corresponds to a constrained problem with $C = \Omega(\hat{\boldsymbol{\theta}})$. The mapping between C and λ need not be one-to-one, and the relation is not generally $C = 1/\lambda$.

For ridge regression, use the augmented quantities above and define $D = \text{diag}(0, 1, \dots, 1)$ so that the intercept is not penalised. The objective

$$J(\tilde{\boldsymbol{\theta}}) = \frac{1}{n} \left\| \tilde{X} \tilde{\boldsymbol{\theta}} - \mathbf{y} \right\|_2^2 + \lambda \tilde{\boldsymbol{\theta}}^\top D \tilde{\boldsymbol{\theta}}$$

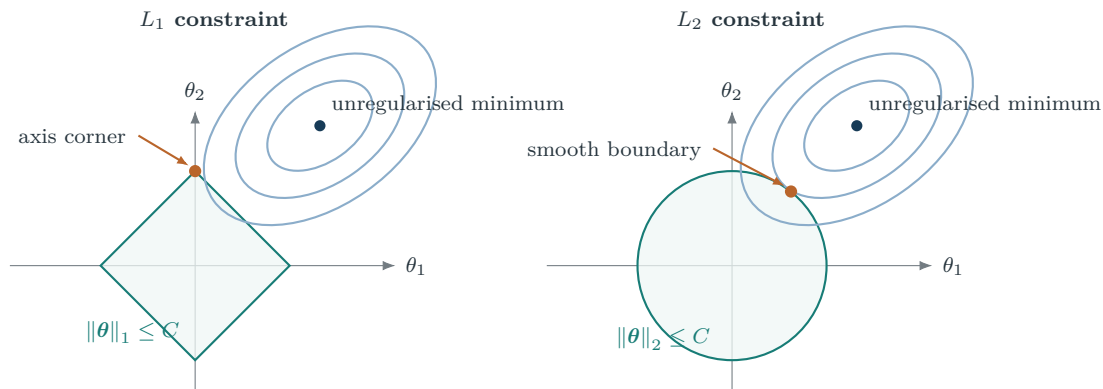


Figure 8: Constrained views of regularisation. Expanding a loss contour from its unregularised minimum until it meets the feasible set gives the constrained solution. An L_1 ball has axis-aligned corners, so contact often sets a coefficient exactly to zero; the smooth L_2 boundary usually produces shrinkage without sparsity. The contact locations shown are schematic rather than universal.

has gradient

$$\nabla J(\tilde{\theta}) = \frac{2}{n} \tilde{X}^\top (\tilde{X} \tilde{\theta} - \mathbf{y}) + 2\lambda D \tilde{\theta}.$$

Setting it to zero gives

$$\hat{\tilde{\theta}}_{\text{ridge}} = (\tilde{X}^\top \tilde{X} + n\lambda D)^{-1} \tilde{X}^\top \mathbf{y}.$$

For $\lambda > 0$ and $n > 0$, $\tilde{X}^\top \tilde{X} + n\lambda D$ is positive definite and invertible: the data term identifies the intercept, while the penalty is positive on every non-zero slope vector.

6.3 Holdout Validation

Development data are commonly divided into a training set and a validation set, with a separate test set reserved for final evaluation. For each hyperparameter setting h , fit $\hat{f}_h$ on the training set and compute

$$\hat{L}_{\text{val}}(h) = \frac{1}{m} \sum_{i=1}^m \ell(y_i^{\text{val}}, \hat{f}_h(\mathbf{x}_i^{\text{val}})).$$

After selecting h , the model may be refitted on the combined training and validation data. The test set is then evaluated once; it must not be used for further tuning.

Suppose h and the fitted model are fixed before validation, the validation observations are iid, and the per-example loss has conditional variance σ_ℓ^2 . Define the risk of this fixed fit as $R(\hat{f}_h)$. Conditional on the training data,

$$\mathbb{E}[\hat{L}_{\text{val}}(h) \mid \mathcal{D}_{\text{train}}] = R(\hat{f}_h), \quad \text{Var}(\hat{L}_{\text{val}}(h) \mid \mathcal{D}_{\text{train}}) = \frac{\sigma_\ell^2}{m}.$$

The standard error therefore decreases at rate $m^{-1/2}$. The notation $O(m^{-1/2})$ describes a scale of sampling fluctuation, not a deterministic identity of the form $\hat{L}_{\text{val}} = L \pm O(m^{-1/2})$. If the same validation set selects among many hyperparameter settings, the smallest observed validation loss is subject to selection bias and cannot replace an independent test estimate.

6.4 K-Fold Cross-Validation

Partition the development data into disjoint folds $F_1, \dots, F_K$. For fold k , fit $\hat{f}_h^{(-k)}$ on the other $K - 1$ folds and evaluate on F_k :

$$\hat{L}_{\text{CV}}(h) = \frac{1}{K} \sum_{k=1}^K \frac{1}{|F_k|} \sum_{i \in F_k} \ell(y_i, \hat{f}_h^{(-k)}(\mathbf{x}_i)).$$

Each observation serves as validation data once. This procedure directly evaluates fits trained on approximately $\frac{K-1}{K}n$ observations, not the final model refitted on all development data.

Stratified folds preserve approximate class proportions in classification. Time-series observations must not be randomly shuffled; forward-chaining folds prevent future observations from predicting the past. If model selection and an approximately unbiased performance estimate are both required, nested cross-validation uses inner folds for selection and outer folds for evaluation.

6.5 Early Stopping

During iterative training, the training loss can continue to decrease after the validation loss begins to rise. If θ_t denotes the parameters after update t , the stopping time can be selected by

$$t^* \in \arg \min_t \hat{L}_{\text{val}}(\theta_t), \quad \hat{\theta} = \theta_{t^*}.$$

Training should restore the parameters from the best validation checkpoint rather than keep only the parameters at the final update. A patience window tolerates short fluctuations caused by mini-batch noise. Since the number of updates is selected through validation performance, early stopping is a form of model selection and implicit regularisation. The test set must not be monitored after every epoch.

6.6 Correlation in Model Averaging

Let $f_1, \dots, f_K$ be base-model predictions at a fixed input, with average

$$\bar{f}_K = \frac{1}{K} \sum_{k=1}^K f_k.$$

In general,

$$\text{Var}(\bar{f}_K) = \frac{1}{K^2} \left(\sum_{k=1}^K \text{Var}(f_k) + 2 \sum_{i < j} \text{Cov}(f_i, f_j) \right).$$

If every model has variance σ_f^2 and every pair has correlation ρ , this reduces to Equation (1). Variance falls as $1/K$ only when $\rho = 0$; for $\rho > 0$, its limit is $\rho\sigma_f^2$. Random forests randomise candidate features partly to reduce correlation among trees. Averaging does not automatically reduce bias; bagging mainly reduces variance for unstable learners.

7 Model Evaluation and Class Imbalance

An evaluation metric is meaningful only after the task, positive class, and decision costs have been specified. Define evaluation metrics before inspecting test results, and select decision thresholds on validation data.

7.1 The Confusion Matrix and Basic Metrics

For a positive class $Y = 1$, the four counts are true positives TP , false positives FP , false negatives FN , and true negatives TN . They define

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \\ \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} = \text{TPR} &= \frac{TP}{TP + FN}, \\ \text{Specificity} &= \frac{TN}{TN + FP}, \\ F_1 &= \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}. \end{aligned}$$

Precision is the fraction of predicted positives that are correct. Recall is the fraction of positive examples that are correctly identified. The F_1 score is their harmonic mean and does not use TN . If a denominator is zero, software libraries may return zero, an undefined value, or a warning; a report should state the convention used.

Accuracy under class imbalance

Consider a test set of 1000 observations with only 60 positives. If $TP = 40$, $FP = 10$, $FN = 20$, and $TN = 930$, then

$$\text{Accuracy} = 0.97, \quad \text{Precision} = 0.80, \quad \text{Recall} = \frac{2}{3}, \quad F_1 \approx 0.727.$$

The 97% accuracy conceals that the classifier misses one third of the positive examples.

7.2 Decision Thresholds and Costs

Many classifiers first return a score $s(\mathbf{x})$ or an estimated probability $\hat{p}(Y = 1 \mid \mathbf{x})$, then apply a threshold τ :

$$\hat{y}_\tau = \mathbb{I}\{s(\mathbf{x}) \geq \tau\}.$$

Lowering τ generally increases both TP and FP . Recall therefore rises, while Precision may rise or fall. A threshold of 0.5 is natural only under particular calibration and cost assumptions; it is not universally optimal.

If a false negative costs C_{FN} and a false positive costs C_{FP} , choose the threshold on validation data by minimising

$$C(\tau) = C_{FN}FN(\tau) + C_{FP}FP(\tau).$$

Ranking quality, probability calibration, and performance at a fixed threshold are distinct properties. AUC assesses ranking. Calibration curves or the Brier score assess probability calibration. Confusion-matrix metrics assess a particular decision rule.

7.3 ROC, Precision–Recall, and AUC

Varying τ gives

$$\text{TPR}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}, \quad \text{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}.$$

The ROC curve plots TPR against FPR. AUROC is the area under this curve. For continuous scores, it is also the probability that a randomly selected positive receives a higher score than a randomly selected negative, with half credit for a tie. AUROC is unchanged by a strictly increasing transformation of the scores and does not assess probability calibration.

The precision–recall (PR) curve plots Precision against Recall. The expected Precision of a random ranking equals the positive-class prevalence. Under severe class imbalance, the PR curve often shows the effect of false positives more directly than the ROC curve. AUPRC and AUROC both summarise ranking across thresholds; neither establishes that a particular deployment threshold meets a cost or recall requirement. Their numerical values are also not directly comparable across datasets with different class prevalence.

7.4 Macro and Micro Averaging

For C classes, treat class c as positive and all other classes as negative to obtain a class-specific metric M_c . Macro averaging weights classes equally:

$$M_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C M_c.$$

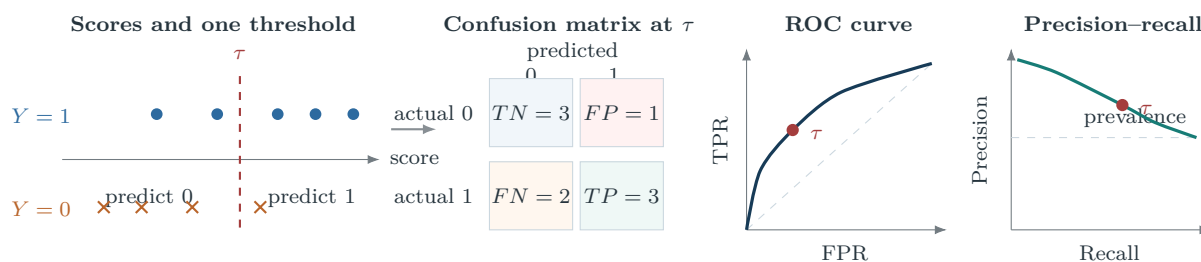


Figure 9: A threshold converts scores into predicted labels and therefore fixes one confusion matrix. That matrix gives one ROC point (FPR, TPR) and one precision-recall point (Recall, Precision). Sweeping the threshold traces both curves; the random-ranking baseline for Precision equals the positive-class prevalence.

It therefore exposes weak performance on rare classes. Weighted macro averaging uses class frequencies:

$$M_{\text{weighted}} = \sum_{c=1}^C \frac{n_c}{n} M_c.$$

Micro averaging first pools the class-specific counts. For example,

$$\text{Precision}_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c (TP_c + FP_c)}.$$

In single-label multiclass classification, where every observation has exactly one true and one predicted class, micro Precision, micro Recall, and micro F_1 all equal Accuracy. Macro averaging gives each class equal weight; micro averaging gives each observation equal weight.

7.5 Regression Metrics

Let $e_i = \hat{y}_i - y_i$ be the residual. Common regression metrics are

$$\begin{aligned} \text{MAE} &= \frac{1}{n} \sum_{i=1}^n |e_i|, \\ \text{MSE} &= \frac{1}{n} \sum_{i=1}^n e_i^2, \\ \text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}, \\ R^2 &= 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \end{aligned}$$

MAE and RMSE have the same units as the target. MSE and RMSE penalise large residuals more strongly; MAE is less sensitive to outliers. The coefficient R^2 compares a model with the baseline that always predicts the sample mean, and it can be negative on unseen data. The appropriate metric depends on the error costs and noise structure.

7.6 Imbalanced Data and Training-Only Resampling

When the minority class is rare, Accuracy can be dominated by the majority class. Stratified sampling can preserve approximate class proportions across the training, validation, and test sets, unless time order or grouping imposes a stricter split. Training-time responses include class-weighted losses, undersampling, oversampling, and SMOTE.

Undersampling discards some majority examples. It reduces computation but may remove useful information. Random oversampling duplicates minority examples. It retains the majority data but can

overfit repeated observations. SMOTE selects nearby minority examples and interpolates in feature space:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \gamma(\mathbf{x}_j - \mathbf{x}_i), \quad \gamma \sim \text{Uniform}(0, 1).$$

This construction assumes that line segments in the feature space are meaningful. Direct application is inappropriate for untreated categorical features, tightly constrained variables, or neighbours whose connecting segment crosses a class boundary.

Boundary for resampling

Split the data first, then resample only the training set. During cross-validation, fit scaling, neighbour searches, and SMOTE separately inside each training fold; leave the validation fold at its original distribution. Neither the validation set nor the test set should be resampled, separately or together with the training set. Doing so changes the evaluation distribution and leaks information into the fitted pipeline.

Oversampling and class weighting alter the class prior seen during fitting, so the resulting scores need not remain calibrated probabilities. If deployment requires probabilities or a risk-based threshold, calibrate and select the threshold on an unresampled validation set.

8 Feature Representation and Dimensionality Reduction

Learning algorithms operate on numerical vectors, while raw observations may be table rows, time series, images, or text. A feature map

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^d, \quad \mathbf{x} = \phi(\text{raw observation})$$

determines which differences are available to the model. Feature engineering constructs new variables, feature selection retains a subset of existing variables, and feature extraction changes the coordinate system. Any fitted part of these operations must use training data alone.

8.1 Consequences of High Dimension

Section 2.3 derives why neighbourhoods become sparse as dimension grows. Irrelevant variables can also add noise to distances and decision boundaries, but high dimension does not inevitably cause overfitting: effective dimension, regularisation, sample size, and structural assumptions all matter. Feature scaling and every other fitted transformation use training-set statistics, which are then applied unchanged to validation and test data.

8.2 Feature Selection

Filter methods score variables on the training set without repeatedly fitting the final predictor. For discrete variables, the mutual information between feature X_j and label Y is

$$I(X_j; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}.$$

The value is zero when X_j and Y are independent, and a larger value indicates greater statistical dependence. Its estimate depends on sample size and, for continuous features, on the density or discretisation estimator. Removing highly correlated features can reduce redundancy, but two correlated variables may still contribute different predictive information. Correlation alone is not a sufficient deletion rule.

Wrapper methods evaluate feature subsets through model validation. Recursive Feature Elimination repeatedly fits a model and removes its currently weakest feature. It is computationally expensive, and the entire selection procedure must remain inside each cross-validation training fold. Embedded methods perform selection during fitting; L_1 regularisation, for example, can set coefficients to zero. Any label-dependent selection performed before the train-validation split leaks label information into evaluation.

8.3 PCA: An Unsupervised Variance Objective

Let $X_c \in \mathbb{R}^{n \times d}$ be the centred data matrix, with sample covariance

$$S = \frac{1}{n} X_c^\top X_c.$$

The first principal direction solves

$$\mathbf{w}_1 \in \arg \max_{\|\mathbf{w}\|_2=1} \text{Var}(X_c \mathbf{w}) = \arg \max_{\|\mathbf{w}\|_2=1} \mathbf{w}^\top S \mathbf{w}.$$

Introduce a Lagrange multiplier:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top S \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1).$$

The stationarity condition is

$$\nabla_{\mathbf{w}} \mathcal{L} = 0 \implies S \mathbf{w} = \lambda \mathbf{w}.$$

The objective value equals the eigenvalue, so $\mathbf{w}_1$ is an eigenvector associated with the largest eigenvalue. Subsequent directions maximise variance subject to orthogonality to the earlier directions. If $W_r = [\mathbf{w}_1, \dots, \mathbf{w}_r]$ contains the top r directions, the projected representation is

$$Z = X_c W_r \in \mathbb{R}^{n \times r}.$$

The explained-variance ratio of these components is

$$\frac{\sum_{j=1}^r \lambda_j}{\sum_{j=1}^d \lambda_j}.$$

PCA does not use labels and preserves linear directions of large sample variance. Large variance need not be useful for classification. Features with large numerical scale can also dominate the covariance matrix, so the choice between centring alone and prior standardisation depends on units and the modelling objective. Principal components are uncorrelated under the sample covariance model; orthogonality does not imply statistical independence in general. The centre, scale, and projection matrix must all be fitted on the training set.

8.4 LDA: A Supervised Separation Objective

Suppose the training data contain C classes. Let n_c and $\boldsymbol{\mu}_c$ be the size and mean of class c , and let $\boldsymbol{\mu}$ be the overall mean. Define the within-class and between-class scatter matrices by

$$S_W = \sum_{c=1}^C \sum_{i: y_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^\top,$$

$$S_B = \sum_{c=1}^C n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top.$$

For a one-dimensional projection $z = \mathbf{w}^\top \mathbf{x}$, Fisher's objective is

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top S_B \mathbf{w}}{\mathbf{w}^\top S_W \mathbf{w}}.$$

Maximising this ratio separates projected class means while limiting the spread within each class. A stationary direction satisfies the generalised eigenvalue problem

$$S_B \mathbf{w} = \lambda S_W \mathbf{w}.$$

A multidimensional projection uses the generalised eigenvectors with the largest eigenvalues. Since $\text{rank}(S_B) \leq C - 1$, LDA provides at most $C - 1$ discriminative directions. In high-dimensional, small-sample settings, S_W may be singular; replacing it by $S_W + \gamma I$ gives a regularised problem.

PCA maximises overall variance, whereas LDA uses labels to maximise class separation. The objective above defines Fisher's dimensionality-reduction projection. Using LDA as a generative classifier adds the assumption that class-conditional distributions are approximately Gaussian with a shared covariance matrix.

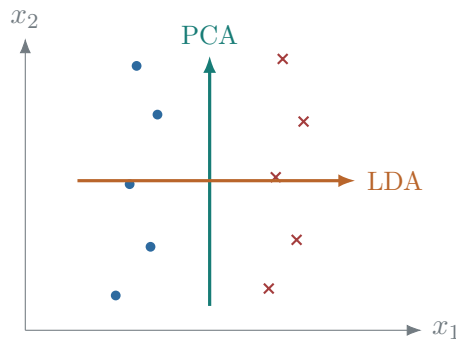


Figure 10: PCA selects directions of high total variance. LDA selects directions that maximise the ratio of between-class to within-class scatter. Their objectives are different.

8.5 Tabular and Temporal Features

Tabular modelling often joins records from several sources by entity and time. Useful aggregates include historical counts, sums, means, extrema, and elapsed times. Every aggregate must be available at the prediction time. A churn model predicting next month, for example, cannot use a transaction total that is finalised only after that month ends. Categorical variables can be one-hot encoded. High-cardinality features require explicit control of dimensionality, and the category dictionary must be built from training data.

Time-series features often use lags and rolling statistics:

$$x_t^{(\text{lag},k)} = y_{t-k}, \quad x_t^{(\text{mean},w)} = \frac{1}{w} \sum_{r=1}^w y_{t-r}.$$

The window must end at $t - 1$, or at the latest value genuinely available when the prediction is made. A periodic variable with period T can be represented by

$$\sin \frac{2\pi t}{T}, \quad \cos \frac{2\pi t}{T},$$

which places the start and end of the cycle next to each other in feature space. Validation for temporal data must preserve chronological order.

8.6 Image Features

A colour image is a tensor of shape $H \times W \times C$, with $C = 3$ for RGB data. A global colour histogram records the frequency of colour ranges but discards their spatial locations. Edge features describe local changes in intensity. For a greyscale image I , horizontal and vertical differences can be estimated by discrete kernels:

$$G_x = K_x * I, \quad G_y = K_y * I, \quad M = \sqrt{G_x^2 + G_y^2}.$$

Thresholding M produces an edge map. Direction histograms, regional statistics, or a lower-dimensional projection of the edge map can then serve as features. Hand-designed colour and edge descriptors encode explicit assumptions; convolutional networks learn local filters from the training objective. Both approaches must use normalisation parameters estimated from training images.

8.7 Text Features

Text is tokenised by a fixed rule, and the training corpus defines a vocabulary $V = \{v_1, \dots, v_d\}$. Bag-of-Words represents document D as a fixed-length vector:

$$x_j = \text{count}(v_j, D), \quad j = 1, \dots, d.$$

The entries may instead be binary indicators or normalised frequencies. This representation records which terms occur but discards word order.

TF-IDF reduces the influence of terms that occur in many documents. Let the corpus contain N documents and let df_j be the number containing term v_j . One smoothed convention is

$$\begin{aligned}\text{tf}_{dj} &= \frac{\text{count}(v_j, D_d)}{\sum_k \text{count}(v_k, D_d)}, \\ \text{idf}_j &= \log \frac{N+1}{\text{df}_j+1} + 1, \\ x_{dj} &= \text{tf}_{dj} \text{idf}_j.\end{aligned}$$

Textbooks and software packages use different conventions for term frequency, smoothing, and the logarithm base. These definitions must be checked before numerical values are compared.

An n -gram treats a consecutive sequence of n tokens as one term. Unigrams discard local order. Bigrams can distinguish phrases such as “not good” and “good”, but they create a larger and sparser vocabulary. Stemming truncates word forms by rule; lemmatisation uses lexical or grammatical information to recover a base form. Stop-word removal is task dependent, since frequent terms such as negations can carry label information. The vocabulary, document frequencies, and filtering rules must be fitted on the training corpus only.

9 From the Perceptron to Multilayer Networks

A linear model reduces an input to a score and predicts from that score. A neural network retains the affine computation but composes several layers, placing a nonlinear function between successive affine transformations.

9.1 The perceptron as a single neuron

For binary labels $y \in \{-1, +1\}$, the perceptron computes

$$a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

and applies a sign activation:

$$\hat{y} = \text{sign}(a(\mathbf{x})).$$

Section 4.3 gives the update rule, loss, and convergence condition. As a network unit, the perceptron exposes the limitation that motivates hidden layers: a sign applied to one affine score still produces a linear decision boundary.

9.2 Multilayer perceptrons

In a multilayer perceptron (MLP), each hidden layer l first applies an affine transformation and then an elementwise activation function:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad (2)$$

$$\mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}), \quad (3)$$

with $\mathbf{a}^{(0)} = \mathbf{x}$. The vector $\mathbf{z}^{(l)}$ contains the pre-activations of layer l , and $\mathbf{a}^{(l)}$ is its output.

The nonlinear activation is essential. If every layer is affine, then two successive layers can still be written as one affine map:

$$\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} = \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}.$$

Adding affine layers alone does not enlarge the class of functions represented by the model.

On a compact input domain, a one-hidden-layer network with sufficient width and a suitable non-polynomial activation can approximate any continuous target function arbitrarily closely. This universal approximation result is a statement about representational capacity; it does not guarantee an efficient width, successful optimisation, or generalisation beyond the training distribution.

Common activation functions and their derivatives are

$$\begin{aligned}\text{sigmoid}(z) = \sigma(z) &= \frac{1}{1 + e^{-z}}, & \sigma'(z) &= \sigma(z)(1 - \sigma(z)), \\ \tanh(z) &= \frac{e^z - e^{-z}}{e^z + e^{-z}}, & \frac{d}{dz} \tanh(z) &= 1 - \tanh^2(z), \\ \text{ReLU}(z) &= \max\{0, z\}, & \text{ReLU}'(z) &= \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}\end{aligned}$$

ReLU is not differentiable at $z = 0$; implementations select a conventional subgradient there, commonly zero. Sigmoid and tanh are smooth but saturate for inputs of large magnitude. ReLU has derivative one on the positive half-line and zero on the negative half-line.

9.3 Output layers and loss functions

The output transformation and loss function must match the prediction task.

Regression. For a real-valued target, a linear output $\hat{\mathbf{y}} = \mathbf{z}$ can be paired with squared error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2.$$

The factor $1/2$ simplifies the derivative and does not change the minimiser.

Binary classification. Let $p = \sigma(z)$ denote the probability of the positive class and let $y \in \{0, 1\}$. The binary cross-entropy is

$$\mathcal{L}_{\text{BCE}} = -y \log p - (1 - y) \log(1 - p).$$

Mutually exclusive multiclass classification. If each sample belongs to exactly one of K classes, the output layer uses Softmax:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K. \quad (4)$$

The resulting values satisfy $p_k \geq 0$ and $\sum_k p_k = 1$. Stable implementations set $m = \max_j z_j$ and evaluate Softmax using $z_k - m$. This shift leaves the probabilities unchanged and prevents overflow in the exponential.

For a one-hot target, or more generally for $y_k \geq 0$ and $\sum_k y_k = 1$, the multiclass cross-entropy is

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log p_k. \quad (5)$$

Multilabel classification. When several labels may be present simultaneously, the classes should not share one softmax normalisation. The model instead computes $p_k = \sigma(z_k)$ independently for each class and sums the K binary cross-entropies.

9.4 The Softmax-cross-entropy gradient

The Jacobian of Softmax is

$$\frac{\partial p_k}{\partial z_j} = p_k (\mathbb{I}[k = j] - p_j), \quad (6)$$

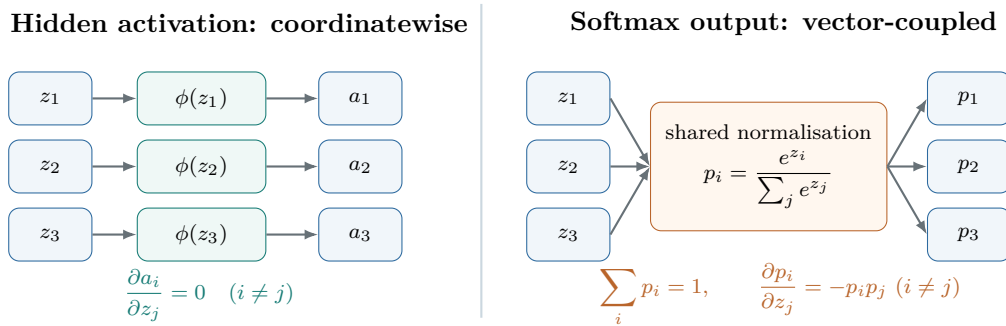


Figure 11: Hidden-layer activations such as ReLU act coordinate by coordinate: changing z_j directly changes only a_j . Softmax uses a shared denominator, so changing one logit generally changes every class probability.

where $\mathbb{I}[k = j]$ equals one when $k = j$ and zero otherwise. Substitution into the derivative of cross-entropy gives

$$\begin{aligned}
 \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_j} &= - \sum_{k=1}^K y_k \frac{\partial p_k}{\partial z_j} \\
 &= - \sum_{k=1}^K y_k (\mathbb{I}[k = j] - p_j) \\
 &= p_j \sum_{k=1}^K y_k - y_j \\
 &= p_j - y_j.
 \end{aligned}$$

Thus,

$$\nabla_{\mathbf{z}} \mathcal{L}_{\text{CE}} = \mathbf{p} - \mathbf{y}. \quad (7)$$

The gradient $\mathbf{p} - \mathbf{y}$ results from the composition of softmax and cross-entropy, rather than from softmax alone.

9.5 Forward and backward computation in a two-layer network

Consider a K -class network with one hidden layer of width m :

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}, \quad (8)$$

$$\mathbf{h} = \phi(\mathbf{z}^{(1)}), \quad \mathbf{h} \in \mathbb{R}^m, \quad (9)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)} \mathbf{h} + \mathbf{b}^{(2)}, \quad \mathbf{W}^{(2)} \in \mathbb{R}^{K \times m}, \quad (10)$$

$$\mathbf{p} = \text{softmax}(\mathbf{z}^{(2)}), \quad \mathcal{L} = - \sum_{k=1}^K y_k \log p_k. \quad (11)$$

Backpropagation applies the chain rule in reverse computational order. The output error signal follows from Equation (7):

$$\boldsymbol{\delta}^{(2)} := \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} = \mathbf{p} - \mathbf{y}.$$

Since $\mathbf{z}^{(2)} = \mathbf{W}^{(2)} \mathbf{h} + \mathbf{b}^{(2)}$, the output-layer gradients are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \boldsymbol{\delta}^{(2)} \mathbf{h}^\top, \quad (12)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \boldsymbol{\delta}^{(2)}. \quad (13)$$

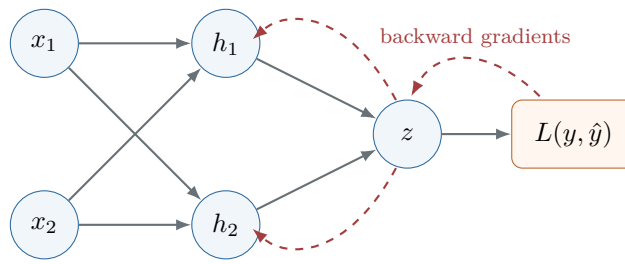


Figure 12: Backpropagation starts at the loss and applies the chain rule along the reversed computation graph. Activations stored during the forward pass are reused to obtain parameter gradients.

The matrix in Equation (12) has shape $K \times m$, matching $\mathbf{W}^{(2)}$.

The gradient with respect to the hidden representation is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \left(\mathbf{W}^{(2)} \right)^\top \boldsymbol{\delta}^{(2)}.$$

Passing this gradient through the elementwise activation gives

$$\boldsymbol{\delta}^{(1)} := \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \left[\left(\mathbf{W}^{(2)} \right)^\top \boldsymbol{\delta}^{(2)} \right] \odot \phi'(\mathbf{z}^{(1)}), \quad (14)$$

where $\odot$ denotes elementwise multiplication. The first-layer gradients are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top, \quad (15)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \boldsymbol{\delta}^{(1)}. \quad (16)$$

For a mini-batch of size B , the usual update averages the sample gradients:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{1}{B} \sum_{i=1}^B \frac{\partial \mathcal{L}_i}{\partial \mathbf{W}^{(l)}}.$$

The same recurrence extends to any number of layers. For $\boldsymbol{\delta}^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)}$,

$$\boldsymbol{\delta}^{(l)} = \left[\left(\mathbf{W}^{(l+1)} \right)^\top \boldsymbol{\delta}^{(l+1)} \right] \odot \phi^{(l)'}(\mathbf{z}^{(l)}), \quad (17)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}. \quad (18)$$

Backpropagation evaluates the chain rule efficiently by caching forward-pass intermediates and reusing them during the reverse pass.

9.6 Saturation and vanishing gradients

Gradients in a deep network contain products of many Jacobians. For a network with L elementwise activation layers,

$$\frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{a}^{(0)}} = \mathbf{D}^{(L)} \mathbf{W}^{(L)} \dots \mathbf{D}^{(1)} \mathbf{W}^{(1)}, \quad \mathbf{D}^{(l)} = \text{diag}(\phi^{(l)'}(\mathbf{z}^{(l)})).$$

If the norms of most factors are below one, the product decays rapidly with depth and gradients near the input approach zero. Repeated factors with norms above one can instead produce exploding gradients.

Sigmoid satisfies

$$0 < \sigma'(z) \leq \frac{1}{4},$$

and its derivative approaches zero as $|z|$ grows. The derivative of $\tanh$ also approaches zero in its saturated regions. ReLU reduces this source of decay on its positive half-line, where its derivative is one, but has zero derivative on its negative half-line. A unit that remains negative may therefore stop updating. ReLU neither has derivative one everywhere nor guarantees that gradients cannot vanish.

Initialisation should keep activation and gradient scales from changing too rapidly across layers. Xavier initialisation is commonly paired with approximately symmetric activations such as $\tanh$, while He initialisation is commonly paired with ReLU. Recurrent networks face the same issue across time steps; gated state updates help control gradient flow over long sequences.

10 Convolutional and Recurrent Networks

Fully connected layers do not encode spatial or temporal structure. Nearby pixels in an image are often related, while a position in a sequence may depend on earlier positions. Convolutional networks use local connectivity and spatial parameter sharing. Recurrent networks share parameters across time and propagate a hidden state along the sequence.

10.1 A convolutional layer computes cross-correlation

Let the input be

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}.$$

For a kernel of spatial size $K_h \times K_w$, with C_{in} input channels and C_{out} output channels, the complete weight tensor has shape

$$\mathbf{K} \in \mathbb{R}^{K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}}.$$

The pre-activation at spatial location (i, j) and output channel o is

$$Z_{i,j,o} = b_o + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=1}^{C_{\text{in}}} K_{u,v,c,o} \tilde{X}_{iS_h+u, jS_w+v, c}, \quad (19)$$

where S_h, S_w are the strides and $\tilde{\mathbf{X}}$ is the padded input. Equation (19) does not reverse the kernel horizontally and vertically, so the operation is formally cross-correlation. Machine-learning software and texts conventionally call it convolution.

Each output channel has one scalar bias b_o . One output channel corresponds to a three-dimensional filter spanning all input channels, not a filter applied to only one channel. The number of parameters is

$$K_h K_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \quad (20)$$

This count does not grow with the input height or width because the same kernel parameters are reused at every spatial position.

10.2 Kernel size, stride, padding, and output size

With padding P_h, P_w and strides S_h, S_w , the output dimensions are

$$H_{\text{out}} = \left\lfloor \frac{H + 2P_h - K_h}{S_h} \right\rfloor + 1, \quad (21)$$

$$W_{\text{out}} = \left\lfloor \frac{W + 2P_w - K_w}{S_w} \right\rfloor + 1. \quad (22)$$

These expressions assume dilation one. Kernel size specifies the local receptive field of one operation; stride specifies the displacement between operations; padding adds rows and columns around the boundary. For an odd-sized kernel and unit stride, choosing

$$P_h = \frac{K_h - 1}{2}, \quad P_w = \frac{K_w - 1}{2}$$

preserves the spatial dimensions.

For example, a 32×32 input passed through a 3×3 kernel with padding one and stride one remains 32×32 . If only the stride changes to two, then

$$H_{\text{out}} = W_{\text{out}} = \left\lfloor \frac{32 + 2 - 3}{2} \right\rfloor + 1 = 16.$$

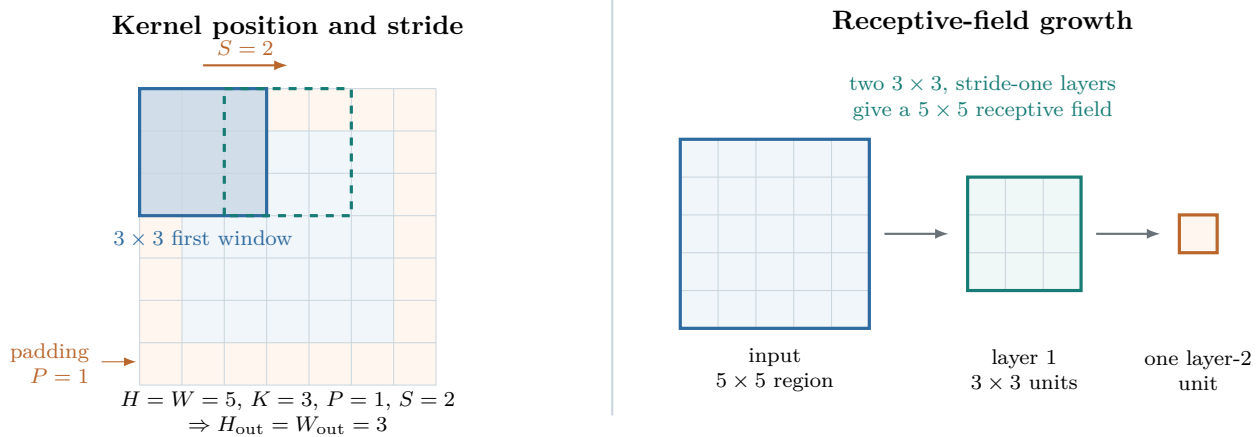


Figure 13: Convolution geometry. On the left, padding enlarges the effective input and stride moves the kernel between sampled positions; the stated values produce a 3×3 output. On the right, one unit after two 3×3 , stride-one convolutions depends on a 5×5 input region, even though each individual operation is local.

10.3 Pooling

Pooling aggregates values within a local window. Max pooling returns the largest value, while average pooling returns the mean. The window size and stride determine the output size through the same calculation as Equations (21)–(22), using the pooling-window parameters.

Standard pooling has no learned parameters. During backpropagation, max pooling routes the gradient only to a location that attained the maximum in the forward pass; average pooling distributes the gradient equally across the window. Pooling reduces spatial resolution and subsequent computation, but discards precise positional information. A strided convolution can instead learn the downsampling operation.

10.4 Recurrent neural networks

Given a sequence $\mathbf{x}_1, \dots, \mathbf{x}_T$, a basic recurrent neural network (RNN) updates its hidden state sequentially:

$$\mathbf{a}_t = \mathbf{W}_x \mathbf{x}_t + \mathbf{U} \mathbf{h}_{t-1} + \mathbf{b}, \quad (23)$$

$$\mathbf{h}_t = \phi(\mathbf{a}_t), \quad (24)$$

$$\mathbf{q}_t = \mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y. \quad (25)$$

The vector $\mathbf{q}_t$ contains the output logits. The matrices $\mathbf{W}_x, \mathbf{U}, \mathbf{W}_y$ are shared across all time steps. An unrolled network appears to contain T cells, but every cell refers to the same parameters. Its parameter count is therefore independent of sequence length.

The state $\mathbf{h}_t$ carries information from earlier positions that is relevant to the current computation. A loss may be applied only at the final position or at every position. The latter case can be written as

$$\mathcal{L} = \sum_{t=1}^T \ell_t(\mathbf{q}_t).$$

10.5 Backpropagation through time

Training an RNN requires backpropagation through time (BPTT). Define

$$\mathbf{e}_t := \frac{\partial \ell_t}{\partial \mathbf{q}_t}, \quad \boldsymbol{\delta}_t := \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t}, \quad \boldsymbol{\delta}_{T+1} = \mathbf{0}.$$

Working backwards from $t = T$,

$$\boldsymbol{\delta}_t = (\mathbf{W}_y^\top \mathbf{e}_t + \mathbf{U}^\top \boldsymbol{\delta}_{t+1}) \odot \phi'(\mathbf{a}_t). \quad (26)$$

The gradients of shared parameters sum the contributions from all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_x} = \sum_{t=1}^T \boldsymbol{\delta}_t \mathbf{x}_t^\top, \quad (27)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}} = \sum_{t=1}^T \boldsymbol{\delta}_t \mathbf{h}_{t-1}^\top, \quad (28)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{t=1}^T \boldsymbol{\delta}_t. \quad (29)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_y} = \sum_{t=1}^T \mathbf{e}_t \mathbf{h}_t^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_y} = \sum_{t=1}^T \mathbf{e}_t. \quad (30)$$

For a loss applied only at the final position, set $\mathbf{e}_t = \mathbf{0}$ for $t < T$.

The influence of an early state on a later state contains a long product of recurrent Jacobians. In schematic form,

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{s=t+1}^T [\text{diag}(\phi'(\mathbf{a}_s)) \mathbf{U}],$$

with the factors ordered by time. This product may decay to zero or grow rapidly, producing vanishing or exploding gradients. Gradient clipping limits exploding gradients but cannot restore gradients that have already vanished. Truncated BPTT propagates gradients through a finite time window, reducing computation and memory at the cost of discarding longer dependencies.

10.6 Information flow in an LSTM

A long short-term memory network (LSTM) introduces a cell state $\mathbf{c}_t$ and gates that control retention, writing, and readout. One common parameterisation is

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (31)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (32)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c), \quad (33)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (34)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (35)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (36)$$

The forget gate $\mathbf{f}_t$ controls retention of the previous cell state, the input gate $\mathbf{i}_t$ controls writing of the candidate state, and the output gate $\mathbf{o}_t$ controls how much of the cell state enters the hidden representation. Along the direct cell-state path in Equation (34),

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \text{diag}(\mathbf{f}_t),$$

when the gate dependence is held fixed. If the forget gate remains close to one, gradients can travel farther through the additive state update. This mechanism mitigates vanishing gradients but does not guarantee preservation of dependencies of arbitrary length.

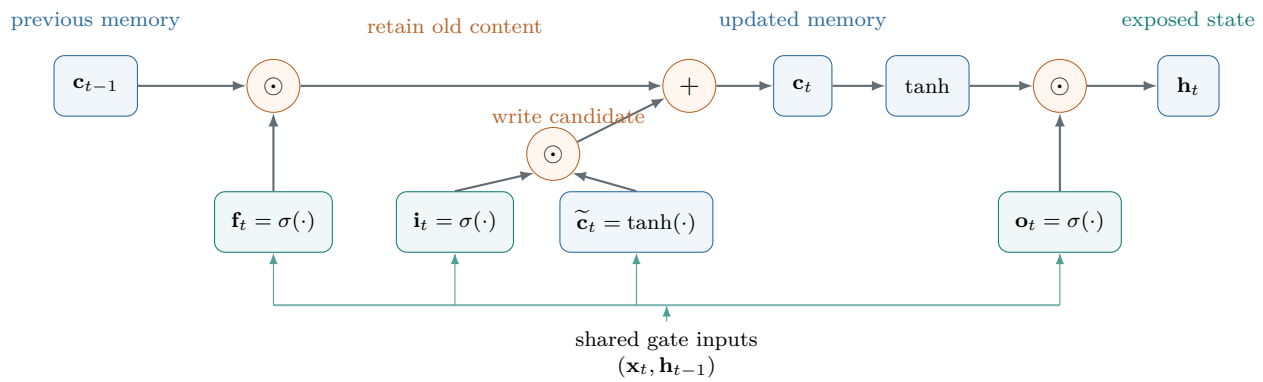


Figure 14: Information flow in an LSTM cell. The forget and input branches form the additive update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$; the output gate exposes $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$. The gates and candidate all depend on $(\mathbf{x}_t, \mathbf{h}_{t-1})$.

10.7 Information flow in a GRU

A gated recurrent unit (GRU) has no separate cell state. The following convention lets larger $\mathbf{z}_t$ write more of the candidate state:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad (37)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z), \quad (38)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \quad (39)$$

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (40)$$

The reset gate $\mathbf{r}_t$ controls how much of the previous state enters the candidate computation. The update gate $\mathbf{z}_t$ interpolates between the previous state and the candidate. Some texts use the opposite convention for $\mathbf{z}_t$; the state-update equation determines its meaning. A GRU has fewer gates and usually fewer parameters than an LSTM with the same hidden width.

11 Unsupervised Learning and Autoencoders

Unsupervised objectives are defined directly on the inputs: k -means minimises within-cluster distance, whereas an autoencoder minimises reconstruction error. Neither objective necessarily recovers groups that coincide with downstream class labels.

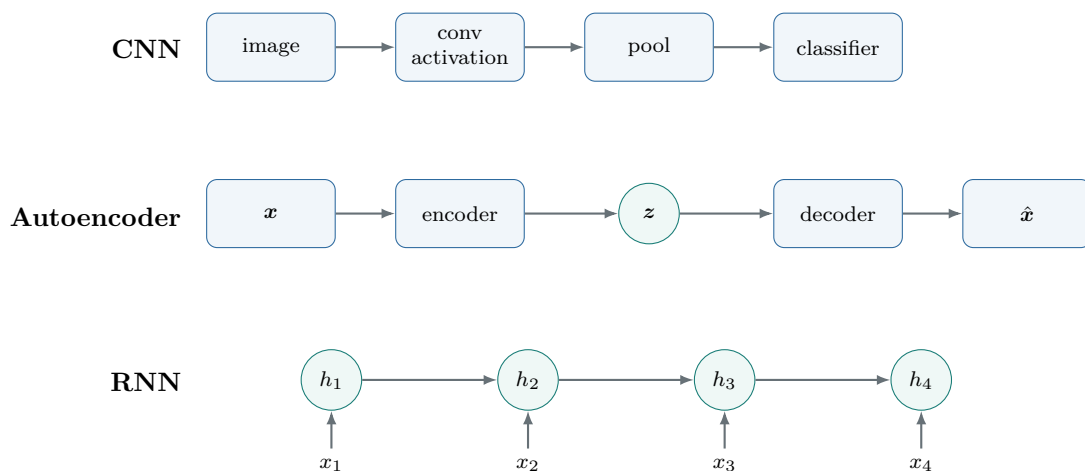


Figure 15: The architectures encode different assumptions: CNNs use spatial locality, RNNs share a state transition across time, and autoencoders learn a bottleneck through reconstruction.

11.1 The K-means objective

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and a chosen number of clusters K , let $c_i \in \{1, \dots, K\}$ be the assignment of sample i , and let $\boldsymbol{\mu}_k$ be the centre of cluster k . K-means minimises the within-cluster sum of squares

$$J(\mathbf{c}, \boldsymbol{\mu}) = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|_2^2. \quad (41)$$

The objective contains both discrete assignments and continuous centroids, so it is normally optimised by alternating between them.

With fixed centroids, each sample is assigned to its nearest centroid:

$$c_i \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2. \quad (42)$$

With fixed assignments, centroid k is updated to the mean of its assigned samples:

$$\boldsymbol{\mu}_k \leftarrow \frac{1}{|C_k|} \sum_{i: c_i = k} \mathbf{x}_i, \quad C_k = \{i : c_i = k\}. \quad (43)$$

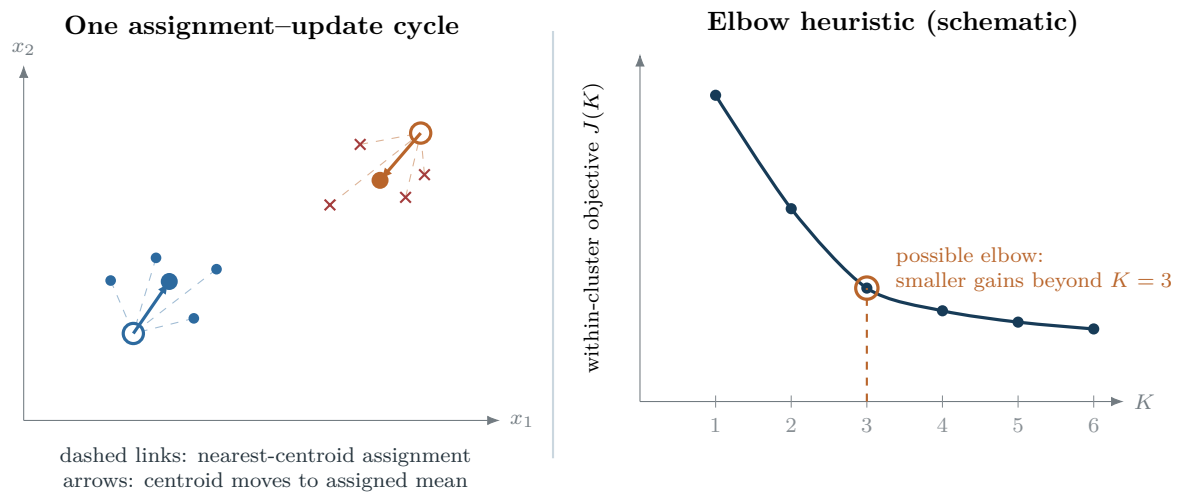


Figure 16: K-means alternates between assigning each sample to its nearest current centroid and replacing each centroid by its assigned mean; each step does not increase the objective. The elbow plot compares the final objective across values of K . A visible bend can guide model selection, but a smooth curve does not determine a unique K .

The mean minimises squared error within a fixed cluster. Both the assignment step and the centroid step leave Equation (41) unchanged or decrease it. Finite stabilisation follows when every assignment change strictly decreases the objective, ties are resolved consistently, and empty clusters are not reinitialised in a way that increases it. The endpoint is a coordinate-wise fixed point, not necessarily a global optimum.

11.2 Initialisation and scope

The initial centroids affect the final solution. Selecting K data points uniformly at random can place several initial centroids close together. Multiple runs with different initialisations are therefore compared by their final objective values; this reduces sensitivity to one poor initialisation but does not guarantee a global optimum.

If a cluster receives no samples, Equation (43) is undefined. An implementation may relocate that centroid to a sample with large current error or reinitialize it. Features should also be scaled before clustering when their numerical ranges differ, because squared Euclidean distance otherwise gives larger-scale features disproportionate influence.

K-means is most suitable for clusters that are approximately spherical, of comparable scale, and separable under Euclidean distance. It is sensitive to outliers and does not directly handle categorical variables or curved, nested clusters. The elbow method examines how J decreases as K grows; if the curve has no clear bend, it does not identify a unique number of clusters.

11.3 The autoencoder objective

An autoencoder consists of an encoder and a decoder:

$$\mathbf{z} = f_{\theta}(\mathbf{x}), \quad (44)$$

$$\hat{\mathbf{x}} = g_{\varphi}(\mathbf{z}). \quad (45)$$

The latent vector $\mathbf{z}$ is an internal representation of the input. The parameters are learned through the reconstruction objective

$$\min_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{x}_i, g_{\varphi}(f_{\theta}(\mathbf{x}_i))) + \Omega(\theta, \varphi). \quad (46)$$

Real-valued inputs are often trained with squared reconstruction error,

$$\ell(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2.$$

Elementwise binary cross-entropy is appropriate when the input components have a Bernoulli interpretation. A Sigmoid output alone is not sufficient reason to assume this statistical model.

An autoencoder is undercomplete when

$$\dim(\mathbf{z}) < \dim(\mathbf{x}).$$

An undercomplete bottleneck limits representational capacity, but it does not by itself guarantee information loss on the data distribution. Data supported on a sufficiently low-dimensional set may still be reconstructed exactly. Under stronger conditions—for example, continuous encoder and decoder maps with data support containing a full-dimensional open set—an exact lower-dimensional code is impossible. If the latent dimension is at least the input dimension and the network has sufficient capacity, the model may approximate the identity map. In either case, low reconstruction error need not indicate a useful representation. Sparsity penalties, denoising objectives, and related regularisation mechanisms can discourage an identity-like solution, especially in an overcomplete model.

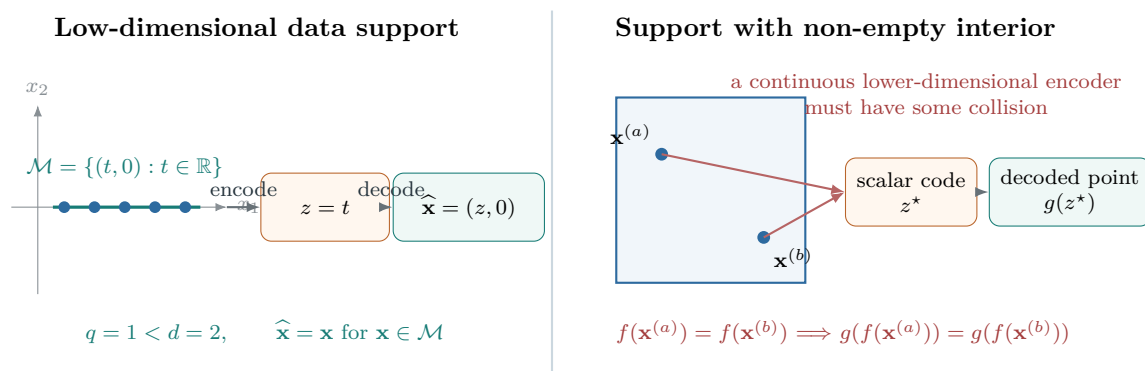


Figure 17: An undercomplete code need not lose information. For $\mathcal{M} = \{(t, 0) : t \in \mathbb{R}\} \subset \mathbb{R}^2$, the maps $f(t, 0) = t$ and $g(t) = (t, 0)$ reconstruct every point exactly with $q = 1 < d = 2$. By contrast, if the data support contains a full-dimensional open set and the encoder is continuous, a lower-dimensional encoder cannot be injective on that support. Bottleneck width constrains capacity but does not by itself determine information loss or representation quality.

11.4 Linear autoencoders and PCA

The relation between a linear autoencoder and principal component analysis holds only under specific conditions. Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be a centred data matrix and let the bottleneck dimension be $q < d$. With a linear encoder, a linear decoder, squared reconstruction error, and no bias, the objective is

$$\min_{\mathbf{W}_e, \mathbf{W}_d} \|\mathbf{X} - \mathbf{X}\mathbf{W}_e\mathbf{W}_d\|_F^2, \quad \mathbf{W}_e \in \mathbb{R}^{d \times q}, \quad \mathbf{W}_d \in \mathbb{R}^{q \times d}. \quad (47)$$

The optimal reconstruction subspace in Equation (47) is the subspace spanned by the first q principal components. Under the additional constraints

$$\mathbf{W}_d = \mathbf{W}_e^\top, \quad \mathbf{W}_e^\top \mathbf{W}_e = \mathbf{I},$$

the reconstruction is the orthogonal projection

$$\hat{\mathbf{X}} = \mathbf{X}\mathbf{W}_e\mathbf{W}_e^\top,$$

and the columns of an optimal $\mathbf{W}_e$ can be chosen as the leading eigenvectors of the data covariance matrix.

Without the orthogonality constraint, latent coordinates are not unique. An invertible change of basis within the same principal subspace can yield the same reconstruction, so individual hidden units need not equal individual principal-component directions. Nonlinear activations, regularisation, or a different reconstruction loss also remove the equivalence with PCA.

11.5 Sparse, deep, and convolutional autoencoders

Sparse autoencoders. A sparse autoencoder may use a wide latent layer while requiring only a small number of units to be active for each input. One option adds $\lambda\|\mathbf{z}\|_1$ to the reconstruction objective, encouraging the encoder to represent each input with relatively few active latent units.

Deep autoencoders. Both encoder and decoder may contain several nonlinear layers. Greater depth can represent nonlinear mappings, but the training problem is non-convex. A lower training reconstruction error does not imply better generalisation or a latent representation that is more suitable for a downstream task.

Convolutional autoencoders. For images, a convolutional encoder preserves local spatial structure and shares parameters across positions. The decoder commonly restores resolution through upsampling followed by convolution, or through transposed convolution. In the latter case, incompatible kernel and stride choices can create uneven overlap and grid artefacts.

11.6 Limits of the compression interpretation

A latent dimension smaller than the input dimension means only that each sample is represented internally by fewer numerical values. It does not determine the size of a compressed file, which also depends on numerical precision, quantisation, entropy coding, and the storage cost of the decoder.

The reconstruction loss determines which information is retained. Squared error favours directions with large numerical variation, which need not carry the semantics required by a classification task. Reconstruction error measures fidelity under the chosen data distribution and loss; it does not establish semantic class structure or a practical compression ratio.